Labo 08 — Questions

Question 1 [Compréhension]

Une équipe construit trois images : `api:1.4.2-dev`, `api:1.4.2-staging` et `api:1.4.2-prod`, chacune avec son `application.yaml` copié dedans. Le déploiement en production échoue sur un paramètre que personne n'avait testé. Expliquez le défaut de méthode, ce qu'il aurait fallu faire, et ce qu'on perd exactement en construisant une image par environnement.

Question 2 [Prédiction]

L'image `api:env` contient `ENV APP_PROFILE=image`. Le fichier `api.env` contient `APP_PROFILE=staging`. Que vaut `APP_PROFILE` dans chacun des cas suivants, et pourquoi l'ordre des options n'a-t-il pas d'importance ?

```
podman run --env-file api.env api:env
podman run --env-file api.env -e APP_PROFILE=prod api:env
podman run -e APP_PROFILE=prod --env-file api.env api:env
APP_PROFILE=shell podman run -e APP_PROFILE api:env
```

Question 3 [Diagnostic]

```
$ podman run -d --name api -p 18080:8080 -e SERVER_PORT=9090 api:1.0
$ curl -s localhost:18080/actuator/health
$ podman logs api
API demarree sur le port 9090 (profil default)
```

L'API tourne, les logs sont normaux, `curl` ne renvoie rien. Qu'est-ce qui cloche, et que dit cet exemple sur la relation entre `EXPOSE`, `-p` et la configuration de l'application ?

Question 4 [Analyse]

Un collègue lance `podman run -d --name api api:1.0 --db-password=Secr3t` « parce que l'application accepte le mot de passe en argument ». Listez **quatre** endroits où ce mot de passe est lisible, avec la commande qui le prouve pour chacun, et classez-les du plus au moins grave.

Question 5 [Analyse]

Le même collègue passe à `-e DB_PASSWORD=Secr3t`. Est-ce mieux ? Listez les endroits où la valeur reste visible, puis expliquez ce que la convention `POSTGRES_PASSWORD_FILE` change concrètement dans `podman inspect`.

Question 6 [Compréhension]

Expliquez comment Spring Boot relie `SPRING_DATASOURCE_URL` à `spring.datasource.url`, ce que fait `SPRING_PROFILES_ACTIVE=prod`, et dans quel ordre Spring Boot départage une valeur présente à la fois dans `application.yaml` (dans le JAR), dans `/config/application.yaml` (monté) et en variable d'environnement.

Question 7 [Diagnostic]

Un Dockerfile contient :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
ENV SPRING_PROFILES_ACTIVE=prod
ENV SPRING_DATASOURCE_PASSWORD=Pr0d!
COPY .env /app/.env
COPY target/api.jar /app/api.jar
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Relevez tout ce qui est contestable, ligne par ligne, et proposez le Dockerfile corrigé en expliquant où chaque information déplacée doit aller.

Question 8 [Prédiction]

```
printf 'S3cr3t' | podman secret create db_password -
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine
```

Que montrent `podman inspect db --format '{{.Config.Env}}'`, `podman exec db ls -l /run/secrets/` et `podman secret inspect db_password` ? Et que faut-il ajouter à `podman secret inspect` pour relire la valeur — ce qui prouve quoi ?

Question 9 [Analyse]

`podman secret` stocke les valeurs encodées en base64 dans votre répertoire utilisateur, sans chiffrement. Un collègue en conclut que « ça ne sert à rien, autant mettre `-e` ». Qu'est-ce que `podman secret` apporte malgré tout par rapport à `-e` ? Et qu'est-ce qu'un gestionnaire de secrets (Vault…) apporte de plus que `podman secret` ?

Question 10 [Diagnostic]

```
$ podman build --secret id=token,src=token.txt -t app .      # OK
$ rm token.txt
$ podman build -t app .                                      # OK aussi ?!
```

Le Dockerfile contient `RUN --mount=type=secret,id=token,required=true npm ci`. Pourquoi le second build réussit-il sans secret, et que faudrait-il pour le voir échouer ? Quelle conséquence pratique pour un jeton qui expire ?

Question 11 [Compréhension]

Comparez `ARG NPM_TOKEN` + `RUN npm ci` et `RUN --mount=type=secret,id=npm_token npm ci` : où le jeton se retrouve-t-il dans chaque cas, et par quelle commande le vérifie-t-on sur l'image produite ? Le multi-stage du labo 05 suffirait-il à protéger l'`ARG` ?

Question 12 [Analyse]

Un fichier `.env` avec les mots de passe de développement est commis dans Git « parce que c'est juste du dev ». Donnez trois raisons distinctes pour lesquelles c'est un problème même en développement, et la pratique de remplacement (fichiers, `.gitignore`, `.dockerignore`, exemple commis).

Question 13 [Analyse]

En production, un secret de base de données a fuité dans un ticket de support (quelqu'un a collé la sortie de `podman inspect`). Décrivez la réponse attendue, étape par étape, et expliquez

pourquoi « supprimer le commentaire du ticket » n'en fait pas partie — puis ce qui, dans l'architecture, rend cette réponse rapide ou lente.